

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационные системы

Лабораторная работа № 1

Информационная система управления фильмами
Вариант 4101

Выполнил:

Чэнь Хаолинъ Р3316

Преподаватель:

Коновалов Арсений Антонович

г. Санкт-Петербург, 2026

Содержание

1	Постановка задачи	2
2	Архитектура и модель данных	3
2.1	Компоненты приложения	3
2.2	Сущности и связи	4
2.3	Удаление и согласованность изменений	5
3	Реализация интерфейса и запросов	6
4	Специальные операции	7
5	Проверка работоспособности	8
6	Исходный код проекта	9
7	Заключение	9

1. Постановка задачи

Требуется разработать информационную систему для хранения и изменения объектов **Movie** и связанных с ними объектов **Coordinates**, **Person** и **Location**. Система должна создавать, показывать по идентификатору, редактировать и удалять фильмы в отдельных интерфейсах, выводить связанные данные и позволять выбирать ранее созданные вспомогательные объекты. Все изменения выполняются на сервере и сохраняются в PostgreSQL.

Главный экран должен показывать все предметные поля фильма в таблице с постраничной загрузкой, сортировкой и фильтрацией строковых значений по полному совпадению. Изменения одной сессии должны автоматически появляться у других авторизованных пользователей. Удаление объекта должно удалять зависящие от него объекты; некорректный ввод должен сопровождаться понятным сообщением.

Отдельный интерфейс должен поддерживать пять операций: удаление по сборам в США, расчёт среднего этого поля, поиск фильмов по начальному фрагменту **tagline**, перераспределение наград между жанрами и дополнительное награждение фильмов, длительность которых превышает порог. Эти действия реализуются в бизнес-логике приложения без хранимых процедур. Для приложения требуются управляемые компоненты Java EE и ORM-провайдер EclipseLink; ограничения предметной области должны соблюдаться как в ORM, так и в базе данных.

Основные поля и их ограничения приведены в таблице ниже.

Таблица 1: Поля объектов и ограничения предметной области

Поле	Тип	Ограничения
Movie		
id	Long	Не null; больше 0; уникальный; генерируется автоматически.
name	String	Не null; непустая строка.
coordinates	Coordinates	Не null.
creationDate	LocalDate	Не null; генерируется автоматически.
oscarCount	int	Больше 0.
budget	long	Больше 0.
totalBoxOffice	Double	Может быть null; при наличии больше 0.
mpaaRating	MpaaRating	Не null.
director	Person	Может быть null.
screenwriter	Person	Дополнительные ограничения в задании не указаны.
operator	Person	Может быть null.
length	Integer	Не null; больше 0.
goldenPalmCount	Long	Может быть null; при наличии больше 0.
usaBoxOffice	int	Больше 0.
tagline	String	Не null.
genre	MovieGenre	Может быть null.
Coordinates		
x	Integer	Не null; не больше 826.
y	Integer	Не null.
Person		
name	String	Не null; непустая строка.
eyeColor	Color	Может быть null.
hairColor	Color	Может быть null.
location	Location	Может быть null.

Поле	Тип	Ограничения
birthday	ZonedDateTime	Не null.
height	Float	Может быть null; при наличии больше 0.
weight	Float	Может быть null; при наличии больше 0.
passportID	String	Не null; уникальный.
Location		
x	Long	Не null.
y	Double	Не null.
name	String	Не null.

Допустимые значения перечислений приведены после UML-диаграммы классов. Поля вспомогательных объектов хранятся в PostgreSQL вместе с фильмами.

2. Архитектура и модель данных

2.1. Компоненты приложения

Серверное приложение собрано в WAR для Payara Micro. Компоненты Jakarta EE выполняют функции Java EE managed beans: REST-ресурсы и фильтр работают с HTTP-запросами, `MovieService` проверяет данные и реализует операции предметной области, `Database` создаёт `EntityManager` и управляет транзакциями. Зависимости внедряются через CDI. Браузер передаёт и показывает данные, не выполняя SQL-запросов и вычислений наград.

JPA-провайдером указан EclipseLink. В интеграционных тестах используется EclipseLink 4.0.5; внутри Payara приложение работает с предоставленной контейнером версией 4.0.1.payara-p3. Сущности, перечисления и конвертер временной зоны находятся в отдельном пакете. Диаграмма пакетов показывает направление зависимостей между частями приложения.

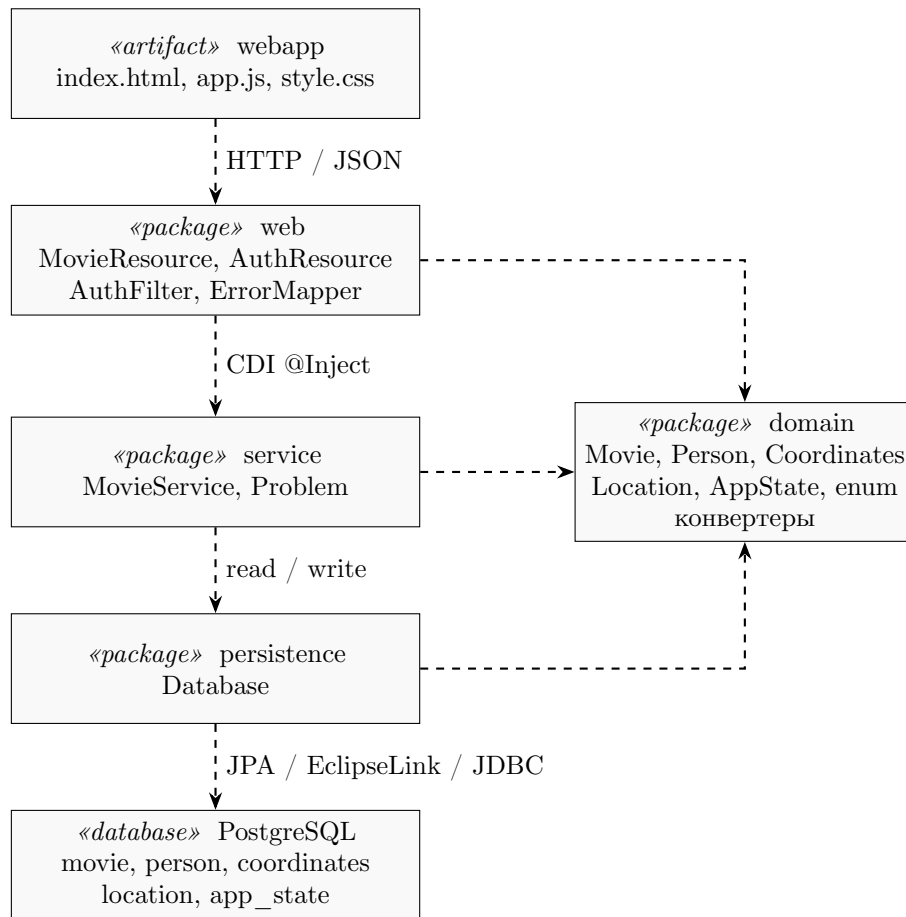


Рис. 1: UML-диаграмма пакетов и внешних компонентов. Зависимости направлены от использующего компонента к используемому.

2.2. Сущности и связи

Фильм обязательно ссылается на координаты и может иметь режиссёра, сценариста и оператора. Каждый человек может ссылаться на место. Один вспомогательный объект допускается использовать для нескольких фильмов. Структуру и кратности связей показывает UML-диаграмма классов.

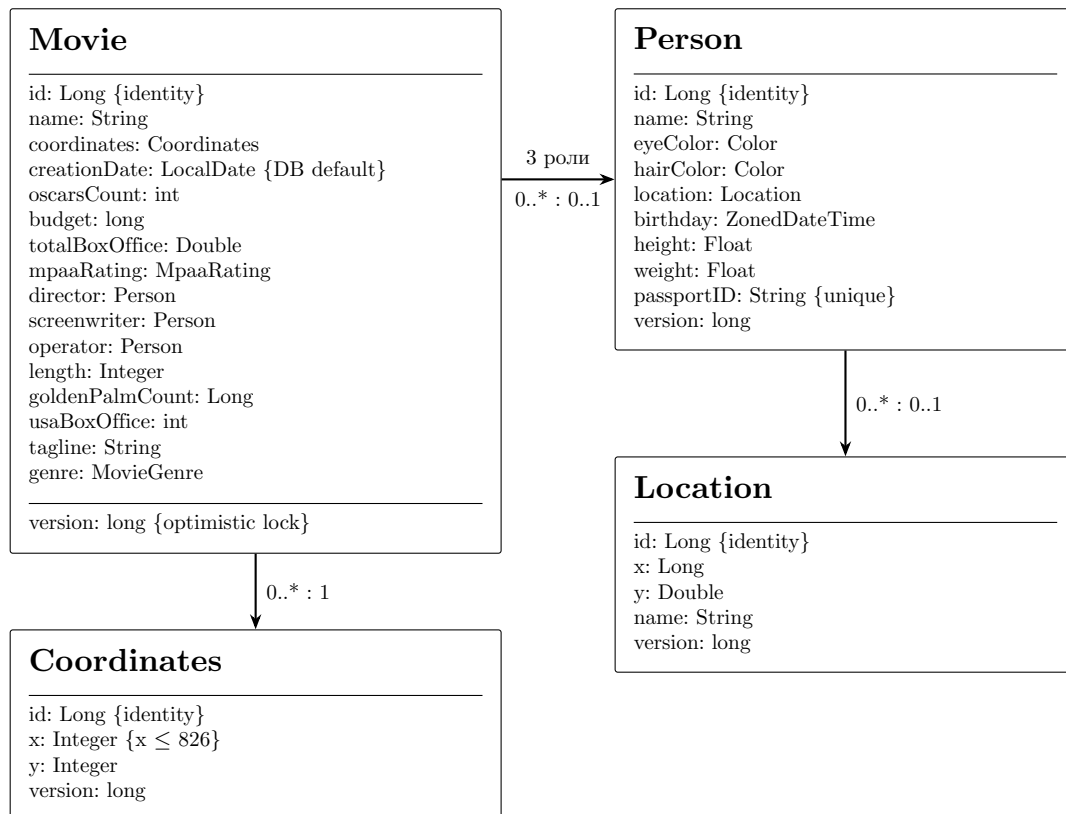


Рис. 2: UML-диаграмма сущностей. Стрелки показывают навигацию; каскад удаления направлен от справочного объекта к зависимым объектам.

Перечисления: MpaaRating {G, PG, PG_13, R, NC_17}, MovieGenre {WESTERN, COMEDY, TRAGEDY, HORROR}, Color {GREEN, BLACK, ORANGE, WHITE, BROWN}. Тип Country {GERMANY, VATICAN, ITALY, NORTH_KOREA} включён в исходники, но не имеет связей в модели варианта.

Идентификаторы формируются в PostgreSQL посредством IDENTITY; дата создания фильма задаётся базой данных при вставке. В Java применяются @NotNull, @Positive, @Max, @Enumerated и связи @ManyToOne. В таблицах повторены обязательность, положительность, диапазон, уникальность паспорта и допустимые значения перечислений с помощью NOT NULL, CHECK, UNIQUE и внешних ключей. Поэтому прямой запрос к БД не позволяет обойти ограничения формы и сервиса.

Время рождения хранится как ISO-строка с часовым поясом через JPA-конвертер: при преобразовании в обычный timestamptz сохранился бы момент времени, но потерялось бы исходное имя зоны. Значения Java Long передаются в JSON десятичными строками, чтобы браузер не округлял большие целые числа. Для атрибутов, у которых задан лишь запрет null, пустая строка разрешена; дополнительные ограничения к исходным полям не добавлялись.

На кафедральном сервере приложение подключено к pg:5432/studs. В схеме учётной записи использованы отдельные префиксы для таблиц приложения и временных таблиц тестов. Таблицы, созданные ранее для других работ, при развёртывании не изменялись. Соответствие имён таблиц задаётся отдельными ORM-отображениями; локальный режим сохраняет обычные имена таблиц.

2.3. Удаление и согласованность изменений

Удаление координат или человека удаляет зависящие от них фильмы; удаление места удаляет связанных людей, а затем их фильмы. Сервис выполняет операции в этом порядке, а внешние

ключи с **ON DELETE CASCADE** обеспечивают то же правило на уровне БД. Удаление одного фильма сохраняет вспомогательные объекты, которыми могут пользоваться другие фильмы.

Перед изменением данных транзакция блокирует служебную строку **AppState** для записи. Это последовательно выполняет групповые операции и защищает их от потери обновлений. У сущностей есть **@Version**; редактирование или удаление по устаревшей версии завершается ответом HTTP 409. Ревизия коллекции увеличивается вместе с изменением данных и откатывается при ошибке. Браузеры запрашивают ревизию раз в две секунды и при изменении обновляют представления, не стирая несохранённые поля открытой формы.

3. Реализация интерфейса и запросов

Меню разделяет фильмы, людей, координаты, места и специальные операции. Создание, просмотр, изменение и подтверждение удаления фильма открываются в отдельных диалогах. Форма позволяет выбрать существующие связанные объекты. В окне просмотра показаны и их атрибуты.

Таблица фильмов содержит отдельную колонку для каждого предметного поля. Постраничный запрос выполняется на сервере, сортировка дополняется идентификатором для устойчивого порядка. Имена полей сортировки и фильтрации проходят разрешённый список, а значения передаются параметрами JPQL. Фильтрация текстовых полей использует полное совпадение и учитывает регистр.

Авторизация использует HTTP-сессию с cookie **HttpOnly**; изменяющие запросы требуют CSRF-токен. При неверном значении поля сервер возвращает структурированную ошибку, которую форма показывает пользователю. Учебное приложение использует общую учётную запись без разделения ролей.

ИТМО / ИНФОРМАЦИОННЫЕ СИСТЕМЫ / 4101

Кинотека

Выйти · student

Фильмы

Люди

Координаты

Места

Специальные операции

Коллекция обновлена · 23:32:00

Фильмы

Добавить

Колонка

Точное значение

Сортировка

Название

Полное совпадение

☐ Включить фильтр

ID

☐ По убыванию

Применить

Действия	ID	Название	Координаты	Дата создания	Число «Оскаров»	Бюджет	Общие сборы	Рейтинг МРАА	Режиссёр	Сценарист	Оператор	Продолжительность
Открыть Изменить Удалить	1	Проверка UI 1790109116491 изменён	#1 · (826; 42)	2026-09-22	5	9223372036854775807	—	PG_13	—	—	—	121

← Назад

Страница 1 из 1 · Объектов: 1

Далее →

На странице 10

Лабораторная работа № 1 · Общая коллекция · Автообновление каждые 2 секунды

Рис. 3: Таблица фильмов в работающем приложении; видна часть прокручиваемых колонок.

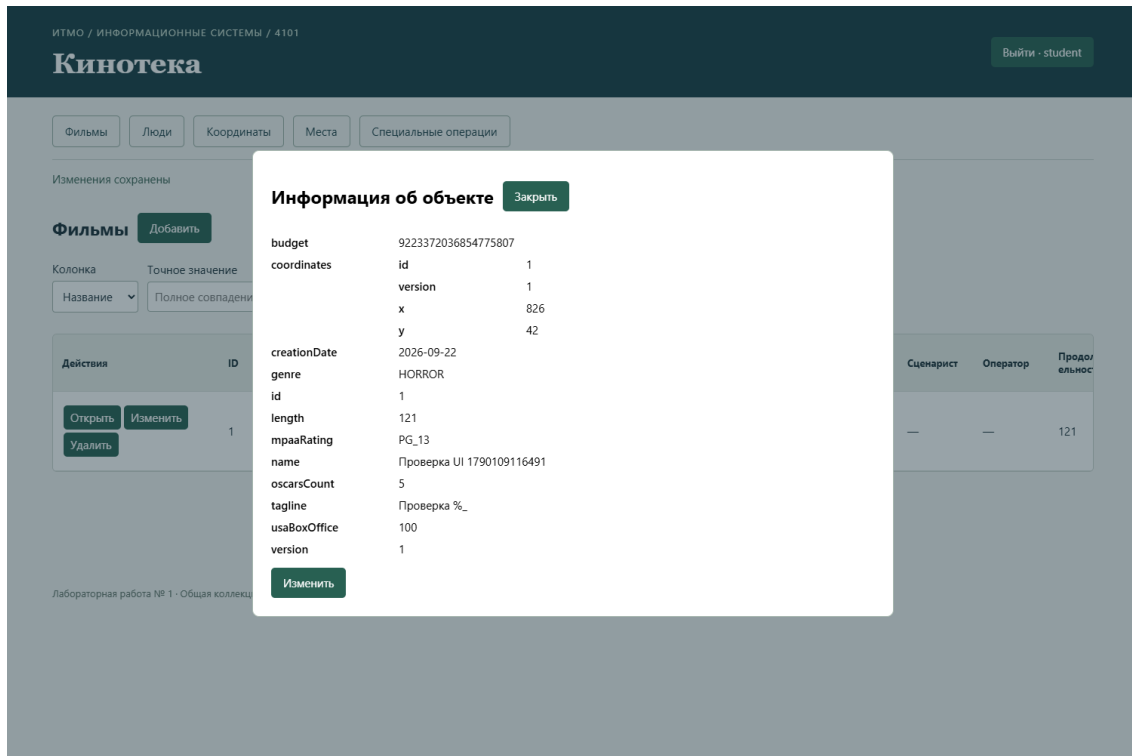


Рис. 4: Отдельное окно просмотра фильма и связанных координат.

4. Специальные операции

Удаление по `usaBoxOffice` выбирает фильмы с точно совпадающим значением и удаляет их в одной транзакции. Среднее вычисляется агрегатом `JPQL AVG`; для пустой коллекции возвращается `null`. Поиск по начальному фрагменту слогана использует `LIKE` с экранированием `%`, `_` и `!`, поэтому введённый фрагмент трактуется буквально. В отличие от него, фильтр главной таблицы требует полного совпадения.

У `oscarsCount` установлено обязательное условие > 0 . Поэтому при перераспределении каждый исходный фильм сохраняет одну награду, а остальные передаются фильмам целевого жанра. Для множества исходных фильмов A и целевых фильмов B , где $n = |B| > 0$, вычисляются

$$S = \sum_{a \in A} (\text{oscars}(a) - 1), \quad q = \left\lfloor \frac{S}{n} \right\rfloor, \quad r = S \bmod n.$$

После присвоения каждому исходному фильму одной награды первые r целевых фильмов по возрастанию `ID` получают по $q + 1$ дополнительной награде, остальные — по q . Общая сумма не меняется, а добавки отличаются максимум на единицу. Например, из исходных значений $(5, 4)$ к целевым $(2, 2)$ переносится $S = 7$; итоговые значения $(1, 1)$ и $(6, 5)$ дают ту же общую сумму 13. Если жанры совпадают или одна из групп пуста, операция отклоняется.

Дополнительное награждение затрагивает только фильмы с `length` строго больше порога. Перед записью проверяется переполнение целочисленного поля. Ошибка у любого фильма откатывает всю операцию, включая уже обработанные записи и ревизию коллекции.

ИТМО / ИНФОРМАЦИОННЫЕ СИСТЕМЫ / 4101

Кинотека

Выйти · student

Фильмы

Люди

Координаты

Места

Специальные операции

Операция выполнена

Специальные операции

Удалить по сборам в США

usaBoxOffice

Удалить совпадения

Средние сборы в США

Для всей коллекции. Пустая коллекция: нет данных.

Рассчитать

Найти по началу слогана

Буквальный префикс

Найти

Перераспределить «Оскары»

Каждый фильм-источник сохраняет 1 награду. Остаток распределяется по возрастанию ID.

Из жанра

WESTERN

В жанр

WESTERN

Перераспределить

Дополнительные награды

Продолжительность строго больше

Число наград

Наградить

Результат

Среднее: 100

Лабораторная работа № 1 · Общая коллекция · Автообновление каждые 2 секунды

Рис. 5: Отдельный экран специальных операций.

5. Проверка работоспособности

Реализация проверена на PostgreSQL с реальным JPA-провайдером, затем собрана и развёрнута на кафедральном сервере Helios под Java 21 и Payara Micro 6.2025.1. На этом сервере команда `Maven clean verify` завершилась успешно: **32 интеграционных теста, 0 ошибок, 0 неудач, 0 пропусков**. Тесты использовали отдельные временные таблицы той же PostgreSQL-базы; после завершения они были удалены. Ниже сгруппированы проверенные свойства.

Таблица 2: Основные группы интеграционных проверок

Группа	Проверенное поведение
Создание и изменение	Генерация ID и даты, допустимые пустые и необязательные поля, сохранение идентичности, запрет изменения служебных полей.
Ограничения	Положительные значения, граница координаты, уникальность паспорта, допустимые перечисления, календарная дата и конечные числа; прямые SQL-попытки нарушения ограничений.
Чтение	Пагинация, сортировка, точный фильтр, обработка необязательных связей и передача максимального значения <code>long</code> .

Группа	Проверенное поведение
Специальные операции	Среднее пустой и непустой коллекций, массовое удаление, буквальный префикс, остаток перераспределения и строгий порог длины.
Транзакции и связи	Откат при переполнении, конкурентные награждения без потерянных обновлений, обновление ревизии после фиксации, направленное каскадное удаление.

Через SSH-туннель запущен сценарий Chrome и API против приложения на Helios. **Все 16 проверок прошли:** отказ неавторизованному запросу и неверному паролю, вход, CSRF, создание и валидация, просмотр связей, обновление и удаление, конфликт устаревшей формы, точная передача большого целого числа, работа специального интерфейса и автоматическое появление изменений во втором браузерном контексте. Ошибок JavaScript не обнаружено. Тестовые фильм и координаты после сценария удалены.

После остановки и повторного запуска Payara подтверждены ответ HTTP 200, работа авторизованного API и доступность пустой коллекции фильмов. Сервер слушает только локальный адрес; для просмотра применяется SSH-туннель. Работоспособность под промышленной нагрузкой не оценивалась.

6. Исходный код проекта

Исходные файлы приложения, SQL-схема и автоматические тесты составляют проект. Листинги кода в отчёт не включены.

Ссылка на репозиторий GitHub:

<https://github.com/utpwtb/ITM0/tree/main/5%20Information%20Systems/Lab/Lab1>

7. Заключение

Разработана информационная система управления фильмами и связанными объектами с хранением в PostgreSQL, JPA-отображением EclipseLink, управляемыми компонентами Jakarta EE и русскоязычным веб-интерфейсом. Реализованы операции создания, чтения, изменения и удаления, точная фильтрация, сортировка и пагинация, каскад зависимостей и пять специальных операций.

Согласованность изменений обеспечивается транзакциями, блокировкой групповых записей и проверкой версий. Взаимодействие двух пользователей подтверждено браузерным сценарием. Сборка и проверки на кафедральном сервере завершились успешно: 32 интеграционных теста и 16 проверок браузера и API. Принятое правило сохранения одной награды у каждого исходного фильма позволяет выполнить перераспределение без нарушения обязательного ограничения поля.